

Computación Concurrente. Práctica 2.

Martínez Mejía Eduardo
Victoria Morales Ricardo Maximiliano
Torres Nava Hazel

10 de septiembre de 2026

1 El programa ColaSecuencial es una implementación de una cola secuencial. Implementa una Cola concurrente utilizando una pool de hilos (ExecutorService). No utilices candados ni synchronized.

Decidimos reutilizar la implementación de ColaSecuencial que nos compartió la profesora, nos parece una implementación interesante y fácilmente implementable.

El código solicitado se encuentra en la ruta de nuestro repositorio:

`practica2/src/ColaConcurrenteNumeros.java`.

Entonces, utilizamos la cola secuencial de la práctica, y en adelante creamos un `ArrayList` futures de `Future<String>` (para guardar los futures asignados a las tareas de los hilos), instanciamos la piscina de hilos con `ExecutorService`, y luego instanciamos la cola. Lo anterior dentro del método `main`, el punto de entrada.

Luego, como se ve en el siguiente código, creamos 100 tareas para nuestra piscina de hilos, estas tareas básicamente consistían de agregar un número a la cola. Cada `Future` devuelto por el método `submit()` fue guardada en la lista futures:

```
for(int i=0; i<100; i++){  
    final int valorActual = i;  
    futures.add(executor.submit(()->cola.enq(valorActual+"")));  
}
```

Después de asignar tareas a la pool de hilos, hicimos `executor.shutdown()` para que los hilos ya no tomaran más tareas, por si las dudas.

Luego, recorrimos la lista `futures` para imprimir el resultado de cada una de sus tareas. Como cada `Future` fue agregado a esa lista en orden, al imprimir cada uno de estos resultados de tareas, la enumeración se imprime en orden, lo que en primera instancia nos hace pensar que se agregaron los elementos en orden. Pero hay que recordar que el método `submit()` para los `Callable` no indica cuándo se ejecutan las tareas de los hilos, sino que indica cuándo se asigno ese hilo con su tarea a un `Future`.

```

/**
Impresion de los resultados de los Future de la lista de Future
en el orden en el que se asignaron las tareas
**/
System.out.println("Imprimiremos_a_los_resultados_de_los_hilos_segun_el_" +
    "orden_en_el_que_se_asignaron_a_sus_tareas,_pero_el_" +
    "contenido_de_la_impresion_de_la_cola_mostrara_el_verdadero_orden_" +
    "en_el_que_se_ejecutaron_los_hilos.");

for(int i=0; i<futures.size(); i++){
    while(!futures.get(i).isDone());
    String resultado = futures.get(i).get();
    System.out.println(resultado);
}

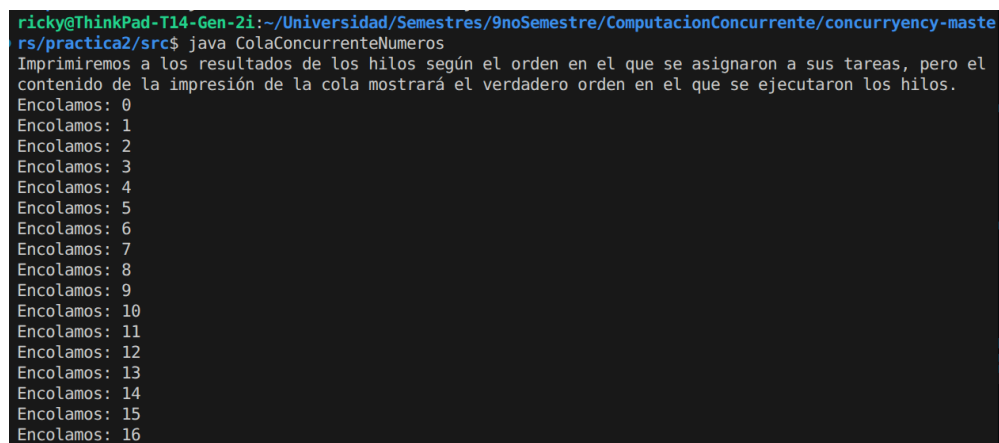
```

Finalmente, imprimimos el estado de la cola con su método `print()`.

2 Ejecuta varias veces tu implementación con diferentes secuencias de llamadas a métodos ¿Tu implementación funciona de acuerdo a lo que se espera de una cola o suceden inconsistencias? Si existe una ejecución así toma una captura pantalla del resultado de tu programa que lo ejemplifique.

De hecho, tenemos dos archivos que tratan de implementar lo que se pide el ejercicio 1. El primer archivo es `ColaSecuencial.java` y el segundo (que es el que se mencionó anteriormente) es `ColaSecuencialNumeros.java`. Con `ColaSecuencial.java` no encontramos inconsistencias, probablemente porque agregamos los elementos a la cola manualmente y no agregamos una cantidad muy grande de elementos. Sin embargo, con `ColaSecuencialNumeros.java` definitivamente sí encontramos inconsistencias.

Las inconsistencias que encontramos en `ColaSecuencialNumeros.java` fueron al momento de imprimir el estado final de la cola después de agregar como 100 elementos a la cola. En la figuras de a continuación, se muestran las salidas y sus explicaciones.



```
ricky@ThinkPad-T14-Gen-2i:~/Universidad/Semestres/9noSemestre/ComputacionConcurrente/concurrency-master/practica2/src$ java ColaConcurrenteNumeros
Imprimiremos a los resultados de los hilos según el orden en el que se asignaron a sus tareas, pero el contenido de la impresión de la cola mostrará el verdadero orden en el que se ejecutaron los hilos.
Encolamos: 0
Encolamos: 1
Encolamos: 2
Encolamos: 3
Encolamos: 4
Encolamos: 5
Encolamos: 6
Encolamos: 7
Encolamos: 8
Encolamos: 9
Encolamos: 10
Encolamos: 11
Encolamos: 12
Encolamos: 13
Encolamos: 14
Encolamos: 15
Encolamos: 16
```

Figura 1: La primera parte de la ejecución, sólo se imprime el resultado de la tarea en el orden en el que fueron asignadas las tareas a los hilos.

Entonces, con las imágenes podemos ver que el programa `ColaConcurrenteNumeros.java` tiene bastantes inconsistencias, esto porque sólomente agregamos números en orden, y el estado de la cola no muestra los números en orden.

```
ricky@ThinkPad-T14-Gen-21:~/Universidad/Semestres/9noSemestre/ComputacionConcurrente/concurrency-master$ java ColaConcurrenteNumeros
La cola final se ve así:
Print
hnull
1
7
0
9
6
5
4
3
10
11
12
13
14
15
16
18
19
```

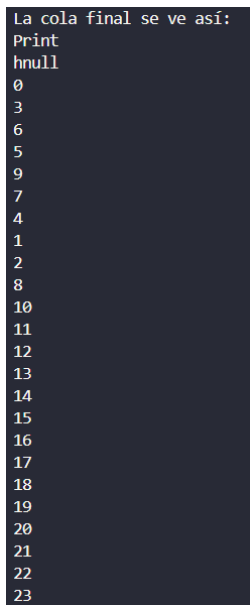
Figura 2: La segunda parte de la ejecución. Se imprime el estado final de la cola, y claramente se ven las inconsistencias.

3 ¿Existen data races o race-conditions? Explica en qué variables y porqué suceden.

- Race Conditions. Es cuando se accede a un mismo recurso compartido, lo que provoca inconsistencias. Claramente existen race conditions porque hay inconsistencias. Y existen específicamente en todas las variables que se modifican y/o acceden en los métodos `enq()` y `deq()` de la cola, la cual todos los hilos comparten. En otras palabras, esto puede ser en el `next` del nodo `head`, en el `next` del nodo `tail`, en el `next` del `newnode`, en el `next` del nodo `last`, entre otros. Esto sucede porque no sabemos en qué orden los hilos están encolando y desencolando en la cola, y de hecho, pueden hacerlo de forma concurrente (solaparse en el tiempo), por lo tanto, pueden modificar variables de forma inconsistente, no hay atomicidad.
- Data Races. Es cuando se accede a un método compartido, lo que provoca inconsistencias. Esto sucede en los métodos `enq()` y `deq()` de la cola. Y si los hilos diferentes el hilo `main` ocuparan el método `print()`, también ese método sería parte de un data race. Esto sucede por la misma razón que el punto anterior, los hilos pueden ejecutar sus tareas o métodos en tiempo solapados, lo que provoca inconsistencias.

4 Utiliza candados y/o synchronized en tu implementación de forma que los métodos enq() y deq() sean una sola sección crítica y contesta: ¿existen inconsistencias?

Sí es inconsistente como se muestra en las siguiente capturas de la ejecución del programa practica2/src/ColaConcurrenteNumerosSynchronized.java.



```
La cola final se ve así:  
Print  
hnull  
0  
3  
6  
5  
9  
7  
4  
1  
2  
8  
10  
11  
12  
13  
14  
15  
16  
17  
18  
19  
20  
21  
22  
23
```

Figura 3: Vemos nuevamente que el orden de asignación de las tareas a los hilos es correcto.

```
PS C:\Users\blood\Documents\ESCUELA\Ciencias Compu\C. Concurrente\Prac
ticas\concurrency-masters\practica2\src> java .\ColaConcurrenteNumero
ssynchronized.java
Imprimiremos a los resultados de los hilos según el orden en el que se
asignaron a sus tareas, pero el contenido de la impresión de la cola
mostrará el verdadero orden en el que se ejecutaron los hilos.
Encolamos: 0
Encolamos: 1
Encolamos: 2
Encolamos: 3
Encolamos: 4
Encolamos: 5
Encolamos: 6
Encolamos: 7
Encolamos: 8
Encolamos: 9
Encolamos: 10
Encolamos: 11
Encolamos: 12
Encolamos: 13
Encolamos: 14
Encolamos: 15
Encolamos: 16
Encolamos: 17
```

Figura 4: Utilizando synchronized en la definición de ambos métodos, los números de la cola final terminan ligeramente revueltos.

5 En una pool de hilos (*ExecutorService*), ¿si utilizamos más hilos equivale a un mayor throughput? Argumenta por qué.

La respuesta general es una gran “depende”, ya que al parecer tiene mucho que ver el tipo de tarea a realizar. Sin embargo, indagando un poco más, parece que la respuesta es que **no necesariamente**. Un mayor número de hilos no equivale automáticamente a un mayor *throughput* (rendimiento o procesamiento de datos), y de hecho, puede llegar a disminuirlo. Esto se debe a que la CPU debe guardar el estado del hilo actual y cargar el del siguiente, por lo que, si hay miles de hilos compitiendo por los recursos, la CPU puede llegar a gastar más tiempo administrando y cambiando entre hilos que ejecutando el código de las tareas. Este proceso se conoce como *context switching* o cambio de contexto.

Otro factor importante es el consumo de memoria, ya que cada hilo en Java reserva su propio espacio de memoria. Por lo tanto, crear demasiados hilos puede provocar un consumo excesivo de memoria e incluso ocasionar un error de tipo `OutOfMemoryError`.

Por lo tanto, aumentar el número de hilos no garantiza un mayor *throughput*. Lo ideal es encontrar una cantidad de hilos adecuada dependiendo del tipo de tarea, los recursos disponibles y la cantidad de núcleos del procesador.

6 Implementación de Tarea y Scheduler para un servidor

El código solicitado se encuentra en la ruta de nuestro repositorio:
`practica2/src/Scheduler.java` y `practica2/src/Tarea.java`

Partiendo del los códigos base compartidos, completamos la definición de la clase **Tarea**, modificando su constructor para agregar las instancias globales de *Semaphore* y *Lock* para controlar el acceso de los hilos.

En **Scheduler**, cada hilo que necesitamos para procesar las peticiones de tareas debe respetar la capacidad del servidor, por lo que creamos un objeto *ExecutorService*, que envía 26 tareas a una *ThreadPool* de 6 hilos, con lo que le podemos pasar a cada uno las referencias compartidas del *Semaphore*, el *Lock*, y el ID de la tarea. De aquí delegamos las tareas al *ExecutorService* y comenzamos la ejecución.

En **Tarea**, *run*, cada hilo realiza lo siguiente:

- Calcula su ID (del 0 al 5) para asignar el tiempo de ejecución correspondiente a cada hilo.
- Espera a adquirir el permiso del semáforo para asegurar que solo entren máximo 3 al mismo tiempo.
- Se implementó una exclusión mutua con el candado asegurando que las tareas del hilo 0 y 2 nunca se ejecuten simultáneamente.

La implementación fácilmente cumple con *justicia* pues Java mismo nos da la opción de cumplirla mediante un parámetro **fair**, establecida como *true*, dentro del constructor de los objetos tipo *Semaphore* y *Lock*, asegurando que el orden de los hilos siempre sea FIFO.

Si no utilizáramos dicho parámetro, podríamos implementar un sistema de turnos dentro de un método *synchronized*: cada hilo tendría una variable entera propia que indique el turno que debe tomar tras asignársele su tarea, así como una variable 'turno' compartida para todos los hilos. De esta forma se podría usar el método *wait()* ciclado para esperar a que el número asignado sea igual al turno global actual, evitando que el hilo entre antes de que deba, saltándose la cola.

Referencias

- [1] G. Komura, *Buenas prácticas de multithreading en la práctica — Edición Java: el estándar en la era de los hilos virtuales*. KomuraSoft LLC, aug 2026.
- [2] Stack Overflow en español. Threads: ¿por qué tener más hilos es mejor? [Online]. Available: <https://es.stackoverflow.com/questions/4921/threads-por-qu%C3%A9-tener-m%C3%A1s-hilos-es-mejor>
- [3] Stack Overflow. How to determine corethreadsize if having multiple threadpools in application? [Online]. Available: <https://stackoverflow.com/questions/76684733/how-to-determine-corethreadsize-if-having-multiple-threadpools-in-application>